

Practical 8

Aim : List and Combo Box Item Verification

Tool: Selenium WebDriver

Automate the process of identifying a list or dropdown box on a web page and determine the total number of items. Validate default selected value and perform selection testing.

1. Import Selenium libraries

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.support.ui import Select
```

```
import time
```

- webdriver → used to open and control the browser.
 - By → used to locate elements on the webpage.
 - Select → used to work with HTML <select> dropdowns.
 - time → used to pause execution for a short time.
-

2. Open Chrome

```
driver = webdriver.Chrome()
```

This creates a Chrome WebDriver object.

It launches the Chrome browser and allows Python/Selenium to control it.

3. Open the website

```
driver.get("https://qaplayground.com/practice/dropdowns")
```

This tells Selenium to open the QA Playground dropdown practice page.

4. Maximize the browser

```
driver.maximize_window()
```

This maximizes the Chrome window.

5. Wait for the page

```
time.sleep(2)
```

The program waits for 2 seconds so that the webpage has time to load.

For professional automation, explicit waits such as `WebDriverWait` are generally preferable to `time.sleep()`.

6. Locate the dropdown

```
element = driver.find_element(By.ID, "fruitSelect")
```

This finds the dropdown using its HTML id.

For example, if the webpage contains:

```
<select id="fruitSelect">
```

then Selenium can locate it using:

```
By.ID, "fruitSelect"
```

The located element is stored in the variable `element`.

7. Create a Select object

```
dropdown = Select(element)
```

This is an important line.

`Select()` allows Selenium to work with a normal HTML `<select>` dropdown.

Now we can use methods such as:

```
dropdown.options
```

```
dropdown.select_by_visible_text()
```

```
dropdown.first_selected_option
```

8. Get all dropdown items

```
options = dropdown.options
```

This retrieves all the <option> elements inside the dropdown.

For example:

Select Fruit

Apple

Banana

Mango

Orange

All of these are stored in options.

9. Count the items

```
print("Total number of items:", len(options))
```

len() counts how many options were found.

For example:

Total number of items: 5

This satisfies the "**determine the total number of items**" requirement.

10. Display all items

```
print("\nDropdown items:")
```

for option in options:

```
    print(option.text)
```

The for loop goes through every dropdown option.

option.text gets the visible text of the option.

Example output:

Dropdown items:

Select Fruit

Apple

Banana

Mango

Orange

11. Check the default selected value

```
default_value = dropdown.first_selected_option.text
```

`first_selected_option` returns the option that is currently selected.

`.text` gets its visible text.

For example, if the dropdown initially shows:

Select Fruit

then:

```
default_value
```

will contain:

Select Fruit

Then:

```
print("\nDefault selected value:", default_value)
```

prints the default value.

This satisfies the **default selected value validation** requirement.

12. Start selection testing

```
print("\nSelection Testing:")
```

This simply displays a heading in the output.

13. Loop through every option

```
for option in options:
```

The program now tests each dropdown item one by one.

For example:

Apple

Banana

Mango

Orange

14. Get the option text

```
text = option.text.strip()
```

This gets the visible text of the current option.

.strip() removes unnecessary spaces from the beginning and end.

15. Skip an empty option

```
if text == "":
```

```
    continue
```

Sometimes a dropdown contains an empty/placeholder option.

If the text is empty, continue skips that option and moves to the next one.

16. Select the option

```
dropdown.select_by_visible_text(text)
```

This selects the dropdown option using its visible text.

For example:

```
dropdown.select_by_visible_text("Apple")
```

selects **Apple**.

Then the loop selects:

Banana

Mango

Orange

one by one.

17. Get the selected value

```
selected_value = dropdown.first_selected_option.text.strip()
```

After selecting an option, this checks which option is actually selected.

For example:

Expected: Apple

Actual: Apple

18. Verify the selection

```
if selected_value == text:
```

```
    print("PASS:", text)
```

```
else:
```

```
    print("FAIL:", text)
```

This compares:

Expected value:

text

with

Actual selected value:

selected_value

If they are equal:

PASS: Apple

If they are different:

FAIL: Apple

This is the **selection verification** part of your assignment.

19. Close the browser

finally:

```
input("\nPress Enter to close...")
```

```
driver.quit()
```

input() keeps the browser open until you press Enter.

Then:

`driver.quit()`

closes the browser and ends the Selenium session.

The finally block ensures that the browser is closed even if an error occurs during the test.

Overall flow of your program

Your program performs exactly these four tasks:

Open website



Locate dropdown



Count total items



Display all items



Check default selected value



Select each item



Verify each selection



Print PASS / FAIL



Close browser

For your practical explanation

"First, I launch Chrome using Selenium WebDriver and open the QA Playground dropdown page. I locate the dropdown using its ID and create a Select object. Using the options property, I get and count all the dropdown items. Then I use first_selected_option to verify the default selected value. Finally, I iterate through all the options, select each option

using `select_by_visible_text()`, and compare the selected value with the expected value. If both values match, the test prints PASS; otherwise, it prints FAIL."

That explanation directly matches the requirement: **identify** → **count** → **validate default** → **perform selection testing**.

Code :

```
from selenium import webdriver

from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

import time
```

```
driver = webdriver.Chrome()
```

```
try:
```

```
    driver.get("https://qaplayground.com/practice/dropdowns")
```

```
    driver.maximize_window()
```

```
    time.sleep(2)
```

```
    # Locate the Fruit dropdown
```

```
    element = driver.find_element(By.ID, "fruitSelect")
```

```
    # Create Select object
```

```
    dropdown = Select(element)
```

```
    # -----
```

```
    # 1. Count total items
```

```
    # -----
```

```
options = dropdown.options
```

```
print("Total number of items:", len(options))
```

```
# -----
```

```
# 2. Display all items
```

```
# -----
```

```
print("\nDropdown items:")
```

```
for option in options:
```

```
    print(option.text)
```

```
# -----
```

```
# 3. Verify default selected value
```

```
# -----
```

```
default_value = dropdown.first_selected_option.text
```

```
print("\nDefault selected value:", default_value)
```

```
# -----
```

```
# 4. Selection testing
```

```
# -----
```

```
print("\nSelection Testing:")
```

```
for option in options:
```

```

text = option.text.strip()

if text == "":
    continue

dropdown.select_by_visible_text(text)

selected_value = dropdown.first_selected_option.text.strip()

if selected_value == text:
    print("PASS:", text)
else:
    print("FAIL:", text)

finally:
    input("\nPress Enter to close...")
    driver.quit()

```

Output:

```

PASS: Apple
PASS: Banana
PASS: Orange

Press Enter to close...
===== RESTART: C:/Users/ditvi/AppData/Local/Programs/Python/Python311/p
Total number of items: 4

Dropdown items:
Select Fruit
Apple
Banana
Orange

Default selected value: Select Fruit

Selection Testing:
PASS: Select Fruit
PASS: Apple
PASS: Banana
PASS: Orange

Press Enter to close... ===== RESTART: C:/Users/ditvi/AppData/Local/Prog

```

Practical 9 :

Aim :

Checkbox Identification and Validation Tool: Selenium WebDriver Write a program to count the total number of checkboxes on a web page, including checked and unchecked counts. Perform dynamic selection and validation.

1. Import required libraries

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
import time
```

- `webdriver` → used to open and control the Chrome browser.
 - `By` → used to identify elements on the webpage.
 - `time` → used to pause the program for a few seconds.
-

2. Open Chrome

```
driver = webdriver.Chrome()
```

This creates the Chrome WebDriver.

In simple words:

Selenium opens the Chrome browser and allows Python to control it.

3. Open the checkbox website

```
driver.get("https://www.lambdatest.com/selenium-playground/checkbox-demo")
```

This opens the LambdaTest checkbox demo page.

4. Use try

try:

The try block contains the main Selenium code.

The advantage is that the finally block will execute even if an error occurs.

5. Maximize the browser

```
driver.maximize_window()
```

This maximizes the Chrome window.

6. Wait for the page to load

```
time.sleep(3)
```

The program waits for 3 seconds.

This gives the webpage some time to load before Selenium starts finding elements.

In real-world automation, WebDriverWait is usually better than time.sleep() because it waits for a specific condition.

7. Find all checkboxes

```
checkboxes = driver.find_elements(  
    By.XPATH,  
    "//input[@type='checkbox']"  
)
```

This is one of the most important parts.

The XPath:

```
"//input[@type='checkbox']"
```

means:

Find all <input> elements whose type is checkbox.

For example, if the HTML contains:

```
<input type="checkbox">
```

```
<input type="checkbox">
```

```
<input type="checkbox">
```

Selenium will find all three.

Why find_elements()?

We use:

```
find_elements()
```

instead of:

```
find_element()
```

because we want to find **multiple checkboxes**.

find_elements() returns a list of matching elements.

8. Count total checkboxes

```
print("Total number of checkboxes:", len(checkboxes))
```

len() counts the number of elements in the list.

For example, if 5 checkboxes are found:

Total number of checkboxes: 5

This satisfies the first requirement:

Count the total number of checkboxes on a web page.

9. Create checked and unchecked counters

```
checked = 0
```

```
unchecked = 0
```

These variables are counters.

Initially:

```
checked = 0
```

```
unchecked = 0
```

As we examine each checkbox, these numbers will increase.

10. Loop through all checkboxes

for checkbox in checkboxes:

This takes each checkbox one at a time.

For example:

Checkbox 1

Checkbox 2

Checkbox 3

Checkbox 4

...

11. Check whether checkbox is selected

if checkbox.is_selected():

is_selected() checks whether the checkbox is currently checked.

If it is checked:

True

If it is unchecked:

False

12. Increase checked count

checked += 1

If the checkbox is checked, increase the checked counter by 1.

For example:

checked = 0

After finding one checked checkbox:

checked = 1

13. Increase unchecked count

else:

unchecked += 1

If the checkbox is not selected, increase the unchecked counter.

So after checking all checkboxes, we might get:

Checked checkboxes: 2

Unchecked checkboxes: 3

This satisfies the second requirement:

Count checked and unchecked checkboxes.

14. Display the counts

```
print("Checked checkboxes:", checked)
```

```
print("Unchecked checkboxes:", unchecked)
```

This displays the results.

Example:

Total number of checkboxes: 5

Checked checkboxes: 2

Unchecked checkboxes: 3

15. Start dynamic selection testing

```
print("\nDynamic Selection Testing:")
```

This just prints a heading.

The next section performs the actual selection testing.

16. Loop through the checkboxes with numbering

```
for i, checkbox in enumerate(checkboxes, start=1):
```

This is slightly different from the earlier loop.

`enumerate()` gives us:

- `i` → checkbox number
- `checkbox` → actual checkbox element

`start=1` means numbering begins at 1.

So the values will be:

i = 1 → Checkbox 1

i = 2 → Checkbox 2

i = 3 → Checkbox 3

17. Select only unchecked checkboxes

if not checkbox.is_selected():

checkbox.click()

This is very important.

First:

checkbox.is_selected()

checks whether the checkbox is already selected.

The word:

not

means the opposite.

Therefore:

if not checkbox.is_selected():

means:

If the checkbox is NOT checked...

Then:

checkbox.click()

clicks the checkbox.

Why do we check first?

Suppose a checkbox is already checked.

If we blindly do:

checkbox.click()

it may become **unchecked**.

We don't want that.

Therefore:

```
if not checkbox.is_selected():
```

```
    checkbox.click()
```

means:

Only click it if it is currently unchecked.

This is called **dynamic selection**.

18. Validate the selection

```
if checkbox.is_selected():
```

After clicking the checkbox, we check it again.

If it is selected:

```
print(f"Checkbox {i}: Selection PASSED")
```

For example:

Checkbox 1: Selection PASSED

If it is not selected:

```
print(f"Checkbox {i}: Selection FAILED")
```

Example:

Checkbox 1: Selection FAILED

So we are not just clicking the checkbox—we are **verifying that the click actually selected it**.

19. Reset the counters

```
checked = 0
```

```
unchecked = 0
```

After the dynamic selection, we count the checkboxes again.

We reset the counters to zero because we are performing a new count.

20. Count the final status

```
for checkbox in checkboxes:
```

```
if checkbox.is_selected():
```

```
    checked += 1
```

```
else:
```

```
    unchecked += 1
```

This loops through every checkbox again.

Now we determine:

- How many are checked?
- How many are unchecked?

Since the previous section selected every unchecked checkbox, ideally all of them should now be checked.

21. Display final results

```
print("\nFinal Status:")
```

```
print("Checked checkboxes:", checked)
```

```
print("Unchecked checkboxes:", unchecked)
```

For example:

Final Status:

Checked checkboxes: 5

Unchecked checkboxes: 0

This confirms that the dynamic selection worked.

22. finally block

finally:

```
    input("\nPress Enter to close...")
```

```
    driver.quit()
```

The finally block executes after the try block.

This line:

```
input("\nPress Enter to close...")
```

keeps the browser open until you press Enter.

Then:

```
driver.quit()
```

closes the browser completely.

Complete flow

Your program works like this:

Start Chrome



Open LambdaTest checkbox page



Find all checkboxes



Count total checkboxes



Check each checkbox



Count checked / unchecked



Select every unchecked checkbox



Verify each checkbox is selected



Count checked / unchecked again



Display final result



Close browser

Code :

```
from selenium import webdriver  
from selenium.webdriver.common.by import By  
import time
```

```
driver = webdriver.Chrome()
```

try:

```
# Open checkbox demo
```

```
driver.get("https://www.lambdatest.com/selenium-playground/checkbox-demo")
```

```
driver.maximize_window()
```

```
time.sleep(3)
```

```
# Find all checkboxes
```

```
checkboxes = driver.find_elements(
```

```
    By.XPATH,
```

```
    "//input[@type='checkbox']"
```

```
)
```

```
print("Total number of checkboxes:", len(checkboxes))
```

```
# -----
```

```
# Count checked and unchecked
```

```
# -----
```

```
checked = 0
```

```
unchecked = 0
```

```
for checkbox in checkboxes:
```

```
    if checkbox.is_selected():
```

```
        checked += 1
```

```
    else:
```

```
        unchecked += 1
```

```
print("Checked checkboxes:", checked)
```

```
print("Unchecked checkboxes:", unchecked)
```

```
# -----
```

```
# Dynamic selection
```

```
# -----
```

```
print("\nDynamic Selection Testing:")
```

```
for i, checkbox in enumerate(checkboxes, start=1):
```

```
    # Select only if currently unchecked
```

```
    if not checkbox.is_selected():
```

```
        checkbox.click()
```

```
    # Validate selection
```

```
    if checkbox.is_selected():
```

```
        print(f"Checkbox {i}: Selection PASSED")
```

```

else:
    print(f"Checkbox {i}: Selection FAILED")

# -----
# Final validation
# -----

checked = 0
unchecked = 0

for checkbox in checkboxes:

    if checkbox.is_selected():
        checked += 1
    else:
        unchecked += 1

print("\nFinal Status:")
print("Checked checkboxes:", checked)
print("Unchecked checkboxes:", unchecked)

finally:
    input("\nPress Enter to close...")
    driver.quit()

```

Output:

```

PRESS ENTER TO CLOSE...
===== RESTART: C:/Users/ditvi/AppData
Total number of checkboxes: 9
Checked checkboxes: 0
Unchecked checkboxes: 9

Dynamic Selection Testing:

```

